Laboratory 8:

Continuous-Time Fourier Properties & Analog Filters
Filter Bank Spectrum Analyzer

Low-Power Spectral Analysis Without the FFT

Lab Duration: 2 hours
Pre-Lab Time Expected: 45 minutes (filter design calculations)
Key Concept: An analog filter bank can measure frequency content directly, without expensive digital signal processing. This is how battery-powered edge sensors stay awake for months.

Contents

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** on all handwritten work, circuit photographs, and MATLAB outputs.

Deliverable (2/2): All filter component calculations must be **handwritten and shown to the TA before you build the circuits**. Using online filter calculators is not accepted; you must derive the component values yourself from the design equations.

Checkpoint (1/7): Before submission, verify that each MATLAB figure includes axis labels with units, a legend, your name/student ID in the title, and clear identification of which data came from hardware vs. software.

1 Real-World Context: Spectral Analysis Without Digital Overhead

You are a Junior Sensor Systems Engineer deployed across three advanced sensing programs at your company. Each program deploys continuous-time analog filters to extract frequency information directly—avoiding the power and latency penalties of digital FFT processing on battery-powered devices. Your role: design and validate the filter banks that power these systems.

1.1 Program 1: Predictive Maintenance for Rotating Equipment (Industrial IoT)

Your company builds ultra-low-power wireless vibration sensors for factories. These sensors clamp onto motors, pumps, and bearing assemblies, monitoring for early signs of mechanical failure.

The challenge: Running a full FFT every 100 ms on a microcontroller draws 50–100 mA continuously. A coin-cell battery (200 mAh) dies in hours. Yet sensors must operate 5+ years in the field to justify installation costs.

Your solution: Design an analog front-end (AFE)—a bank of 4–6 continuous-time bandpass filters that listen to specific frequency bands 24/7 (drawing only microamps). Each filter monitors one "symptom" band: bearing wear (8–12 kHz), imbalance (1x and 2x line frequency), blade-pass frequency, etc. When a specific band spikes above a threshold, only then does the CPU wake to perform deeper analysis. Total power consumption: < 50 µA continuous. Regulatory requirement: achieve $\pm 10\%$ frequency accuracy and identify frequency components within 50 Hz resolution.

1.2 Program 2: Power Grid Frequency Stabilization (Electrical Utilities)

Your company supplies grid-monitoring equipment to utility companies. These devices detect electrical anomalies—harmonics, imbalances, and frequency drift—that herald equipment failures or cascading blackouts.

The challenge: Power grid fundamental frequency is 60 Hz (50 Hz internationally), but harmonics—integer multiples (120 Hz, 180 Hz, 240 Hz, etc.)—indicate nonlinear loads, transformer saturation, and power quality degradation. Utilities need real-time spectral monitoring at monitoring stations worldwide with limited IT infrastructure and no reliance on cloud processing.

Your solution: Design a continuous-time filter bank that simultaneously measures 60 Hz (fundamental), 120 Hz (2nd harmonic), 180 Hz (3rd harmonic), 240 Hz (4th harmonic), and beyond. Each filter's output is fed to a low-bandwidth ADC. No FFT required—just selective analog measurement. Regulatory requirement: < 1 Hz frequency error and harmonic amplitude accuracy $\pm 5\%$ to meet NEMA and IEC 61000 power quality standards. The entire AFE consumes less power than a single microcontroller FFT.

1.3 Program 3: Wireless ECG Monitoring for Cardiac Patients (Biomedical)

Your company manufactures wearable cardiac monitors for remote patient monitoring. These devices detect arrhythmias and alert clinicians to life-threatening conditions.

The challenge: ECG signals contain the heartbeat (0.5–5 Hz fundamental, with harmonics), electrical noise (60 Hz power line interference), and muscle artifacts (20–200 Hz). A patient wearing the device all day drains a battery-powered system quickly if continuous high-rate digital processing is running.

Your solution: Design a dual-band analog filter: a low-pass filter (cutoff 40 Hz) passes the ECG heartbeat, while a notch filter (60 Hz, high Q) rejects power-line interference without affecting the ECG. The output is sampled at just 250 Hz (vs. 10 kHz if a digital filter had to suppress interference). Total power: 20 µA AFE plus minimal ADC overhead. Clinical requirement: achieve $\geq$ 95% sensitivity and specificity for arrhythmia detection, with latency $<$ 2 seconds for alert generation.

1.4 The Common Design Principle: Analog Spectral Analysis

All three programs rely on the same core principle: **use continuous-time filters to measure spectral content directly, without expensive digital transformation**.

- **Battery-powered devices:** Analog AFE stays active 24/7 for 5 years. Digital FFT would kill the battery in weeks.
- **Real-time event detection:** Analog thresholding (comparator) triggers CPU wake instantly—no digital latency.
- **Scalability:** Analog solutions do not require firmware updates. Ship thousands of devices with fixed frequency coverage.
- **Regulatory compliance:** Utilities and medical devices operate under strict standards (NEMA, IEC, FDA). Analog provides deterministic, auditable measurement.

In this lab, you will build a prototype filter bank, measure its frequency response, and validate against FFT theory using Parseval's Theorem—proving that your analog design captures the same spectral information as a full digital transform, but at a fraction of the power cost.

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Design active op-amp bandpass filters:** Calculate resistor and capacitor values for specified center frequencies and quality factors using standard design equations.
2. **Understand continuous-time frequency response:** Sketch the magnitude response $|H(j\omega)|$ and phase response $\angle H(j\omega)$ of your filters and predict how they will respond to input signals.
3. **Build a multi-stage analog circuit:** Assemble 3–4 op-amp filter stages on a breadboard, accounting for loading effects and impedance matching.
4. **Measure spectral content without FFT:** Read the peak amplitude from each filter's output and reconstruct an approximate frequency spectrum.
5. **Apply Parseval's Theorem:** Verify that the total energy of a signal computed in the time domain equals the total energy computed in the frequency domain (sum of squared Fourier coefficients).
6. **Compare analog vs. digital spectral estimation:** Use MATLAB's FFT to compute the true spectrum, overlay it with your hardware-derived spectrum, and quantify accuracy.
7. **Appreciate real-world sensor trade-offs:** Discuss frequency resolution, measurement bandwidth, power consumption, and component tolerances in the context of the AFE design.

3 Analog Filters and Frequency Response

3.1 Continuous-Time Filter Fundamentals

A continuous-time (CT) filter is characterized by its **frequency response**:

$$H(j\omega) = |H(j\omega)|e^{j\angle H(j\omega)}, \tag{1}$$

where $\omega = 2\pi f$ is the angular frequency (rad/s).

For a bandpass filter, the magnitude response has a peak at the **center frequency** $\omega_c = 2\pi f_c$. The width of the peak (at half-power, or -3 dB) defines the **bandwidth**:

$$BW = f_{high} - f_{low} \quad \text{(Hz)}. \tag{2}$$

The **quality factor** (Q) is the ratio of center frequency to bandwidth:

$$Q = \frac{f_c}{BW}. \tag{3}$$

Higher Q $\Rightarrow$ narrower, more selective filter. Lower Q $\Rightarrow$ wider, less selective.

3.2 Active Bandpass Filter Topology

A simple **active bandpass filter** using an op-amp has the form:

Transfer function:

$$H(s) = \frac{G\omega_c s}{s^2 + \frac{\omega_c}{Q}s + \omega_c^2}, \tag{4}$$

where $s = j\omega$ is the complex frequency, G is the low-frequency gain, $\omega_c = 2\pi f_c$, and Q is the quality factor.

The magnitude response at the center frequency is:

$$|H(j\omega_c)| = G \cdot Q. \tag{5}$$

3.3 Parseval's Theorem: Time-Domain Energy = Frequency-Domain Energy

For a periodic signal $x(t)$ with period T, the **average power** (or energy over one period) is:

Time-domain calculation:

$$E_{\text{time}} = \frac{1}{T}\int_0^T x^2(t)\,dt. \tag{6}$$

Frequency-domain calculation: If $x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty}\left[a_k\cos(k\omega_0 t) + b_k\sin(k\omega_0 t)\right]$, then:

$$E_{\text{freq}} = \frac{a_0^2}{4} + \frac{1}{2}\sum_{k=1}^{\infty}(a_k^2 + b_k^2). \tag{7}$$

Parseval's Theorem states:

$$E_{\text{time}} = E_{\text{freq}}. \tag{8}$$

In your lab, you will verify this by measuring energy in both domains and computing the percent error.

[INSERT DIAGRAM: Bandpass filter magnitude response curve showing center frequency, bandwidth, Q factor, and -3dB points]

4 Pre-Lab Preparation: Filter Design

4.1 Your Filter Bank Assignment

The TA will assign you **three to four target center frequencies**. Common values:

- **Low:** 250 Hz, 500 Hz, 1 kHz
- **Mid:** 1 kHz, 2 kHz, 4 kHz
- **High:** 4 kHz, 8 kHz, 16 kHz

Student Notes: Your assigned filter center frequencies:

Filter 1: $f_{c1} =$ _____ Hz

Filter 2: $f_{c2} =$ _____ Hz

Filter 3: $f_{c3} =$ _____ Hz

Filter 4 (optional): $f_{c4} =$ _____ Hz

4.2 Filter Design Equations

For a standard **Sallen-Key (voltage-controlled voltage source) bandpass topology**, the component values are related to the desired center frequency and Q factor by:

$$\omega_c = 2\pi f_c = \frac{1}{R_1 R_2 C_1 C_2} \tag{9}$$

$$Q = \frac{\pi f_c C_1 (R_1 + R_2)}{2} \tag{10}$$

If you assume **equal resistances** ($R_1 = R_2 = R$) and **equal capacitances** ($C_1 = C_2 = C$):

$$\omega_c = \frac{1}{RC} \quad \Rightarrow \quad RC = \frac{1}{\omega_c} = \frac{1}{2\pi f_c} \tag{11}$$

$$Q = \frac{\pi f_c \cdot C \cdot 2R}{2} = \pi f_c RC = 1 \tag{12}$$

So with equal components, you get $Q = 1$ (a moderately selective filter).

Pre-lab Deliverable (1/3): For each assigned center frequency, select standard resistor and capacitor values from the lab (e.g., R from [1kΩ, 10kΩ, 100kΩ], C from [10nF, 100nF, 1μF]) such that $RC \approx 1/(2\pi f_c)$. Show your calculation for each filter.

Student Notes: Space for handwritten filter design calculations:

4.3 Parseval's Theorem Statement

Pre-lab Deliverable (2/3): Write out the statement of Parseval's Theorem in both its time-domain and frequency-domain forms (Equations 6 and 7). Explain in 2–3 sentences what this theorem means physically.

Student Notes: Handwritten Parseval's Theorem statement and explanation:

Checkpoint (2/7): Show the TA your component calculations and Parseval statement for Checkpoint 1 (CP-1) sign-off.

5 Required Equipment and Software

Table 1. Required Hardware and Software

Category	**Required Items**
Microcontroller	Arduino Uno or Mega, USB cable
Op-amps	3–4 `TL072` or `LM358` op-amps (low-noise preferred)
Passive components	Resistors (assorted 1k to 100k Ω), capacitors (assorted 10nF to 10μF)
Signal source	Arduino DAC/PWM output or benchtop function generator
Measurement	Oscilloscope (20 MHz+), multimeter
Prototyping	Breadboard, jumper wires, $\pm$5V or $\pm$15V power supply for op-amps
Software	Arduino IDE, MATLAB R2021a or later
Provided skeletons	`MysterySignal_Gen.ino`, `Lab08_FilterBankCapture.m`

Arduino Pin Roles

Table 2. Pin Assignments

Signal	**Pin**	**Direction**	**Purpose**
Mystery Signal Out	D11 (PWM)	OUTPUT	Injects test waveform into all filters
Filter 1 Output	A0	INPUT	Reads output of bandpass filter 1
Filter 2 Output	A1	INPUT	Reads output of bandpass filter 2
Filter 3 Output	A2	INPUT	Reads output of bandpass filter 3
Filter 4 Output (opt.)	A3	INPUT	Reads output of bandpass filter 4 (optional)
Serial TX	USB	TX	Streams amplitudes and raw signal to MATLAB

6 Hardware Setup: The Analog Front-End (AFE)

6.1 System Architecture

The lab's AFE consists of:

1. **Arduino Signal Generator:** Outputs a "mystery signal" (a TA-assigned periodic waveform) on D11 via PWM.
2. **Bandpass Filter Bank:** 3–4 op-amp active filters, each tuned to different center frequencies, all receiving the same input signal.
3. **Peak Detector (or ADC direct):** Each filter's output is connected to an Arduino ADC pin to measure peak voltage (or RMS via averaging).
4. **Oscilloscope Probe:** Measures the input waveform and at least one filter output to verify operation.

6.2 Bandpass Filter Circuit

A standard **Sallen-Key non-inverting bandpass topology** uses one op-amp per stage:

Key design elements:

- Input signal goes to a high-pass RC stage (passes high frequencies, blocks DC).
- Output of high-pass drives the non-inverting input of the op-amp.
- Feedback network includes a low-pass RC stage (passes low frequencies in the feedback).
- The combination creates a bandpass response peaked at f_c.

[INSERT OP-AMP BANDPASS SCHEMATIC: Sallen-Key topology showing input RC (high-pass), op-amp, feedback RC (low-pass), output. Label R1, R2, C1, C2, and power supply pins.]

6.3 Multi-Stage Wiring

On your breadboard:

1. **Signal path:** Arduino D11 PWM → RC low-pass filter ($\sim$ 1 kHz cutoff to remove PWM switching) → wires to all four bandpass filter inputs in parallel.
2. **Filter outputs:** Connect each bandpass output to a separate Arduino ADC pin (A0, A1, A2, A3).

3. **Power supply:** All op-amps share the same $\pm 5V$ (or $\pm 15V$) power rails. Bypass capacitors (100 nF) across power pins are critical.
4. **Oscilloscope probe:** Place on the input signal (after the PWM low-pass filter) and sweep to each filter output as needed.

[INSERT BREADBOARD WIRING DIAGRAM: Showing Arduino D11 to low-pass filter, then to 4 parallel bandpass filters, each connected to A0, A1, A2, A3. Include power supply, bypass caps, and ground connections.]

Student Notes: Student wiring checklist:

- ☐ PWM low-pass filter wired and tested
- ☐ All 3–4 bandpass filters built and power-connected
- ☐ Filter outputs wired to A0, A1, A2, A3
- ☐ Bypass capacitors on all power pins
- ☐ Oscilloscope probe ready

Checkpoint (3/7): TA inspects the breadboard wiring. Verify that each op-amp has correct power supply connections and that filter outputs produce visible signals on the oscilloscope.

7 Lab Procedure (In-Lab Execution, 2 Hours)

Target Timeline: CP-1 (10 min) → Hardware wiring (30 min) → Filter tuning (20 min) → Hardware spectral measurement (20 min) → MATLAB analysis (35 min) → wrap-up (5 min).

7.1 Part 1: Pre-Lab Verification

1. Show the TA your handwritten filter component calculations.
2. Show your Parseval's Theorem statement.
3. Receive your "mystery signal" waveform type from the TA (e.g., square wave, sawtooth, pulse).

Checkpoint (4/7): TA verifies pre-lab work and assigns the mystery signal for Checkpoint 1.

7.2 Part 2: Build and Test the Bandpass Filter Bank

1. Using your calculated component values, build the first bandpass filter on the breadboard. Double-check resistor values with a multimeter.
2. Power up the op-amp (apply $\pm 5V$ or $\pm 15V$ to the power pins).
3. Apply a 1 V peak sinusoid at your filter's center frequency (f_c) using an external function generator or Arduino PWM.
4. Measure the output with an oscilloscope. You should see a larger amplitude than at off-resonance frequencies.
5. Tune (if needed): If the peak is far from f_c, slightly adjust the resistor value or capacitor value to shift the center frequency.
6. Repeat for filters 2, 3, and 4.

Student Notes: Filter tuning notes:

Checkpoint (5/7): TA visually inspects the complete breadboard. Verify that the Arduino can read non-zero voltages from A0, A1, A2, A3 when a signal is applied.

7.3 Part 3: Upload Mystery Signal Generator and Measure Filter Outputs

1. Upload the TA-provided `MysterySignal_Gen.ino` to your Arduino. This sketch generates your assigned mystery waveform on D11 at a specified frequency (typically 1 kHz).
2. In the Arduino Serial Monitor (115200 baud), you will see streaming data:

```
ADC_A0, ADC_A1, ADC_A2, ADC_A3, raw_sample_count
```

```
512, 256, 128, 64, 0
530, 280, 130, 70, 1
...
```

3. Allow the sketch to run for at least 100 samples (a few seconds). Watch the ADC values from each filter:
   - The filter connected to the strongest harmonic in your mystery signal will show the largest ADC amplitude.
   - Filters far from the mystery signal's harmonics will show smaller readings (just noise).
4. **Make an educated guess:** Based on which filters have high vs. low readings, make a guess about your mystery signal's shape and dominant frequency. Write this down.

Student Notes: Hardware-based spectrum measurement:

Filter 1 (f_{c1}): ADC amplitude _____ (rough peak value)

Filter 2 (f_{c2}): ADC amplitude _____

Filter 3 (f_{c3}): ADC amplitude _____

Filter 4 (f_{c4}): ADC amplitude _____

My guess for the mystery signal shape: _____

Checkpoint (6/7): TA records your guess (without revealing the answer yet) for Checkpoint 2.

7.4 Part 4: MATLAB Spectral Analysis and Validation

Objective: Capture the raw mystery signal and filter outputs in MATLAB, compare hardware vs. software spectrum, and verify Parseval's Theorem.

Task 1: Capture Data and Reconstruct Hardware Spectrum

The provided `Lab08_FilterBankCapture.m` skeleton opens the serial port, reads the streaming ADC data, and stores it in arrays.

```
% MATLAB: Connect and capture
port = serialport("COM3", 115200);
write(port, "s");  % trigger capture

% Read streaming ADC data (100+ samples)
raw_data = [];
for i = 1:100
    line = readline(port);
    values = str2num(line);
    raw_data = [raw_data; values];
end
```

```
% Extract filter amplitudes
filter_amps = [mean(raw_data(:,1)), mean(raw_data(:,2)), ...
               mean(raw_data(:,3)), mean(raw_data(:,4))];
```

Create a bar chart showing the amplitude (in ADC counts) of each filter output:

```
figure;
f_centers = [f_c1, f_c2, f_c3, f_c4];  % your center frequencies
bar(f_centers, filter_amps);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('ADC Amplitude (counts)', 'FontSize', 12);
title(sprintf('Alice Smith (A123) - Hardware Filter Bank Spectrum'), 'FontSize', 12);
grid on;
```

[INSERT EXAMPLE MATLAB FIGURE: Bar chart showing 4 filters with varying heights (one tall bar for the dominant harmonic, shorter bars for others)]

Student Notes: MATLAB capture notes:

Task 2: Compute True FFT and Overlay

Capture the raw mystery signal (the unfiltered Arduino PWM output after the input RC filter) and compute its FFT:

```
% Assume you've captured the raw signal in array 'raw_signal'
Fs = 1000;  % sampling rate (Hz), match your Arduino configuration
Y = fft(raw_signal);
N = length(raw_signal);
frequencies = (0:N-1) * Fs / N;
magnitude_fft = abs(Y) / (N/2);

% Plot software FFT
figure;
plot(frequencies(1:N/4), magnitude_fft(1:N/4), 'b-', 'LineWidth', 1.5);
```

```
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude (V)', 'FontSize', 12);
title(sprintf('Alice Smith (A123) - True FFT of Mystery Signal'), 'FontSize', 12);
grid on;

% Overlay hardware spectrum (normalized to same scale)
hold on;
bar(f_centers, filter_amps / max(filter_amps) * max(magnitude_fft), ...
    'FaceAlpha', 0.5, 'DisplayName', 'Hardware AFE');
legend('Digital FFT', 'Hardware AFE', 'FontSize', 11);
hold off;
```

[INSERT EXAMPLE MATLAB FIGURE: Overlay plot showing true FFT (smooth curve) and hardware filter bank (bar chart). Mark harmonic peaks.]

Task 3: Verify Parseval's Theorem

Compute the total energy in both time and frequency domains:

```
% Time-domain energy (Parseval)
E_time = sum(raw_signal.^2) / length(raw_signal);

% Frequency-domain energy (sum of squared Fourier coefficients)
% For a real signal, energy is sum of magnitude-squared FFT values
E_freq = sum(magnitude_fft(1:N/2).^2);

% Percent error
pct_error = abs(E_time - E_freq) / E_time * 100;

fprintf('Time-domain energy: %.4f\n', E_time);
fprintf('Frequency-domain energy: %.4f\n', E_freq);
fprintf('Percent error: %.2f %%\n', pct_error);
```

Student Notes: Parseval's Theorem verification:

$E_{\text{time}} =$ _____ (V^2)

$E_{\text{freq}} =$ _____ (V^2)

Percent error: _____ %

Is the error acceptable ($< 10\%$)? _____

Checkpoint (7/7): Show TA all three MATLAB figures and the Parseval verification output for Checkpoint 3 (CP-3). Reveal the actual mystery signal shape and compare with your guess.

8 Post-Lab Analysis and Deliverables

8.1 Required Discussion Questions

Pre-lab Deliverable (3/3): Answer each question in 3–6 complete sentences, supported by your lab data.

1. **AFE Accuracy and Resolution:** Compare your hardware-measured spectrum (filter bank) with the true FFT. Where do they agree, and where do they diverge? Could you improve the AFE by adding more filters at intermediate frequencies?
2. **Parseval's Theorem Validation:** Your time-domain and frequency-domain energy calculations should match (within measurement error). If they don't match within $< 10\%$, what could be wrong? (Consider: spectral leakage, window effects, ADC quantization.)
3. **Power Consumption Trade-Off:** A microcontroller running a continuous FFT (1024-point, every 100 ms) draws 50–100 mA and destroys a coin-cell battery in days. Your AFE draws microamps. But what do you lose? What signals might your AFE miss if it only listens to 4 frequency bands?
4. **Filter Selectivity and Q:** You designed your filters with $Q = 1$ (equal R and C). What would happen if you increased Q to 5 by adjusting capacitor values? Would your AFE become more or less sensitive to vibration at off-resonance frequencies?
5. **Real-World Scenario:** For industrial bearing fault detection, you need to identify when high-frequency vibrations (e.g., 5–10 kHz) spike above baseline. Design a rough AFE for this application: how many filters would you need, at what center frequencies, and why?

Student Notes: Space for discussion answers (or attach separate sheet with name/ID):

8.2 Final Submission Checklist

Before submitting, verify that your submission includes **all of the following**:

- ☐ **Pre-lab work:** Handwritten filter component calculations with name/ID, Parseval's Theorem statement.
- ☐ **Hardware photograph:** Clear image of the breadboard showing all 3–4 bandpass filters, op-amps, power connections, and connections to Arduino.
- ☐ **Mystery signal guess:** Written statement of your guess for the signal shape based *only* on hardware measurements (before seeing the answer).
- ☐ **MATLAB Figure 1:** Hardware filter bank spectrum (bar chart) with axis labels, frequency values, and your name/ID in title.
- ☐ **MATLAB Figure 2:** True FFT of the mystery signal with hardware AFE overlay, showing comparison and any discrepancies.
- ☐ **MATLAB Figure 3:** Parseval energy verification—printed output showing E_{time}, E_{freq}, and percent error calculation.
- ☐ **Discussion answers:** Written responses to five discussion questions with numerical support.

- ☐ **Arduino code appendix:** Print the configuration section of `MysterySignal_Gen.ino` showing signal type and frequency.
- ☐ **MATLAB code appendix:** Print the key analysis sections (data capture, FFT computation, Parseval calculation).
- ☐ **TA sign-off page:** See Table 3 below, completed with TA initials at each checkpoint.

8.3 TA Checkpoint Sign-Off

Table 3. TA Checkpoint Sign-Off (Complete as Lab Progresses)

CP	Requirement	Target Time	TA Initials & Date
CP-1	Pre-lab filter calculations and Parseval statement verified. Mystery signal assigned.	0:00–0:10	_____
CP-2	All 3–4 bandpass filters built, tested, and producing visible oscilloscope signals. ADC readings streamed to Serial Monitor. Hardware-based spectrum guess recorded.	0:10–1:00	_____
CP-3	MATLAB data capture successful. FFT spectrum computed and overlaid with hardware AFE data. Parseval verification shows $E_{\text{time}} \approx E_{\text{freq}}$ (error $< 10\%$).	1:00–2:00	_____

Lab Complete: Once all three checkpoints are signed off and the final submission checklist is complete, your proof-of-concept AFE is accepted. Congratulations—you've just designed the power-critical heart of an industrial sensor.

Appendix A: Continuous-Time Fourier Analysis

A.1 Fourier Series and Energy

For a periodic signal $x(t)$ with period T:

Fourier Series representation:

$$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} \left[a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t) \right], \tag{13}$$

where $\omega_0 = 2\pi/T$.

Parseval's Theorem:

$$\frac{1}{T} \int_0^T x^2(t)\, dt = \frac{a_0^2}{4} + \frac{1}{2} \sum_{k=1}^{\infty} (a_k^2 + b_k^2). \tag{14}$$

The left side is the average power in the time domain. The right side is the energy summed across all frequency components.

A.2 Frequency Response of Filters

The magnitude response at the center frequency of a bandpass filter with gain G and quality factor Q is:

$$|H(j\omega_c)| = G \cdot Q. \tag{15}$$

The half-power bandwidth (where $|H(j\omega)| = |H(j\omega_c)|/\sqrt{2}$) is:

$$BW = \frac{\omega_c}{Q}. \tag{16}$$

Appendix B: Op-Amp Circuit Design

B.1 Sallen-Key Bandpass Configuration

The Sallen-Key non-inverting bandpass filter has the transfer function:

$$H(s) = \frac{G\omega_c s}{s^2 + \frac{\omega_c}{Q}s + \omega_c^2}. \tag{17}$$

For equal-component design (R, C same for both stages):

- Center frequency: $f_c = \frac{1}{2\pi RC}$
- Quality factor: $Q = 1$ (with equal R and C)
- DC gain: $G = 1$ (non-inverting unity-gain topology)

To increase Q, reduce the feedback capacitor:

$$C_{\text{feedback}} = \frac{C}{Q}. \tag{18}$$

B.2 Op-Amp Power Supply

Standard audio-grade op-amps (TL072, OPA2134, LM358) require:

- Dual rail: $\pm 5V$ or $\pm 15V$ (recommended for best signal range)
- Bypass capacitors: 100 nF ceramic + 10 μF electrolytic across each power rail pair
- Common ground: All signal grounds, power grounds, and Arduino GND connected at one point (star grounding)

Appendix C: MATLAB Signal Processing Templates

C.1 Serial Data Import and Averaging

```
% Read streaming ADC data from Arduino
port = serialport("COM3", 115200);
write(port, "c");  % trigger capture command

% Collect multiple samples and average
n_samples = 200;
adc_readings = zeros(n_samples, 4);

for i = 1:n_samples
    line = readline(port);
    values = sscanf(line, '%d,%d,%d,%d');
    adc_readings(i, :) = values(1:4);
end

% Average for each filter
filter_means = mean(adc_readings, 1);
fprintf('Filter 1 average ADC: %d\n', filter_means(1));
fprintf('Filter 2 average ADC: %d\n', filter_means(2));
fprintf('Filter 3 average ADC: %d\n', filter_means(3));
fprintf('Filter 4 average ADC: %d\n', filter_means(4));
```

C.2 Parseval Energy Verification

```
% Time-domain energy
E_time = mean(x_time.^2);  % average power for periodic signal

% Frequency-domain energy (from FFT)
X = fft(x_time);
N = length(x_time);
magnitude_spectrum = abs(X(1:N/2)) / (N/2);

% Energy = sum of squared magnitudes (Parseval for real signals)
E_freq = sum(magnitude_spectrum.^2);

% Normalize to compare
E_freq_normalized = E_freq / (N/2);

% Percent error
error = abs(E_time - E_freq_normalized) / E_time * 100;

fprintf('E_time: %.6f\n', E_time);
fprintf('E_freq: %.6f\n', E_freq_normalized);
fprintf('Error: %.2f %%\n', error);
```

C.3 Spectrum Comparison Plot

```
% Plot FFT and hardware AFE on same figure
figure('Position', [100, 100, 1000, 600]);

% FFT (top subplot)
subplot(2, 1, 1);
Fs = 10000;  % sampling rate
frequencies = (0:N/2-1) * Fs / N;
plot(frequencies, magnitude_spectrum, 'b-', 'LineWidth', 1.5);
xlabel('Frequency (Hz)', 'FontSize', 11);
ylabel('Magnitude (V)', 'FontSize', 11);
title('Digital FFT of Mystery Signal', 'FontSize', 12);
grid on;

% Hardware AFE (bottom subplot)
subplot(2, 1, 2);
f_centers = [500, 1000, 2000, 4000];
filter_amps_normalized = filter_means / max(filter_means) * max(magnitude_spectrum);
bar(f_centers, filter_amps_normalized, 'FaceAlpha', 0.7);
xlabel('Center Frequency (Hz)', 'FontSize', 11);
ylabel('Normalized Amplitude', 'FontSize', 11);
title('Analog Filter Bank (Hardware AFE)', 'FontSize', 12);
grid on;

sgtitle(sprintf('Alice Smith (A123) - Spectrum Comparison'), 'FontSize', 13);
```

Appendix D: Troubleshooting Guide

Hardware Issues

1. **Op-amp outputs are flat (constant voltage).**
   - Verify power supply voltage at the op-amp pins (should be $\pm 5V$ or $\pm 15V$).
   - Check that feedback resistors are connected correctly (compare schematic to breadboard).
   - Try a different op-amp (the one you have may be faulty).
2. **Filter output has no signal or very weak signal.**
   - Verify the input signal (from Arduino PWM) is reaching the first filter stage. Use oscilloscope on the filter input.
   - Check capacitor values: a wrong value (e.g., 100 nF instead of 100 pF) will shift center frequency by 1000x.
   - Ensure the Arduino is actually outputting on D11: use `analogWrite(D11, 128)` to test, measure 2.5 V on an oscilloscope.
3. **Filter peak is not at the designed center frequency.**
   - Tolerance on resistors and capacitors can shift the center frequency by 5–10%. Trim by adjusting capacitor value.
   - Measure actual R and C values with a multimeter; use the measured values in the design equation to predict actual f_c.
   - If peak is much lower than expected, you likely have too much capacitance (e.g., parallel capacitors from previous wiring).
4. **ADC readings from A0, A1, A2, A3 are constant (no variation).**
   - Verify that the Arduino is actually reading the correct pins (check the sketch configuration).
   - Ensure filter outputs are connected to ADC pins before power-up.
   - Use a multimeter to verify there is a changing DC voltage at each filter output.

MATLAB Analysis Issues

1. **FFT shows very small magnitudes (¡ 0.001 V).**
   - Check the normalization: for one-sided FFT, use `magnitude = abs(X) / (N/2)`, not `abs(X) / N`.
   - Ensure your input signal is in volts (not ADC counts or normalized to 0–1).
   - If signal was generated as PWM (0–255), convert to voltage: `x_volts = (x_adc / 127.5 - 1) * 5`.
2. **Parseval error is very large (¿ 30%)**

- Spectral leakage: FFT assumes the signal is periodic within the window. If the capture length is not an integer number of periods, leakage spreads energy across bins.
- Window the signal before FFT: `x_windowed = x_time .* hann(length(x_time))'` reduces leakage.
- ADC quantization: 10-bit ADC introduces noise. A cleaner analog signal yields better agreement.

3. **Hardware and software spectra don't match.**
   - Ensure you're using the same mystery signal for both: hardware filters receive it, MATLAB FFT operates on the same raw capture.
   - Confirm your filter center frequencies are correct (match your pre-lab calculations).
   - If discrepancy is large at one frequency, that filter may be mistuned—remeasure with oscilloscope.

When to Ask for TA Help

- If no op-amp output is visible on oscilloscope after power-up (likely power or feedback wiring issue).
- If Parseval error exceeds 50% (suggests fundamental measurement or data capture problem).
- If you're unable to match any of your filter center frequencies to design predictions (may indicate component labeling error).

References and Inspiration

University Course Inspirations

This lab draws on the combined expertise of several world-class programs in signal processing and circuit design:

- **MIT 6.007 (Applied Electromagnetics):** The op-amp bandpass filter topology and transfer function derivation are rooted in MIT's rigorous treatment of circuit analysis and analog system design. The lab emphasizes the Sallen-Key configuration because it mirrors the pedagogical progression in MIT's circuits curriculum—from first-order RC stages to second-order active filters—while remaining implementable on a breadboard.
- **MIT 6.003 (Signals, Systems, and Inference):** Parseval's Theorem, the continuous-time frequency response analysis, and the energy equivalence between time and frequency domains are cornerstones of MIT's signals curriculum. This lab validates that theory by direct hardware measurement.
- **Georgia Tech ECE 2026 (Real-Time DSP):** The real-world context—battery-powered sensors, low-latency event detection, and power consumption trade-offs—reflects Georgia Tech's strong emphasis on **practical, power-conscious DSP**. The lab motivates why analog filtering avoids the energy cost of FFT computation on microcontrollers.
- **Rice University ELEC 241 (Digital Signal Processing):** Rice's embedded systems and real-time processing philosophy informs the focus on measurable, implementable filter designs rather than theoretical ideal responses. The emphasis on component tolerance, tuning, and hardware validation reflects Rice's hands-on DSP approach.
- **Duke University ECE 280:** This lab bridges foundational continuous-time analysis (time and frequency domains) with practical analog implementation, preparing students for upper-level courses in RF engineering, analog IC design, and sensor systems.

Core References

- Oppenheim, A. V., Willsky, A. S., & Young, I. T. (1997). *Signals and Systems* (2nd ed.). Prentice Hall.
  - **Chapters 4–5:** Continuous-time Fourier analysis, frequency response, and Parseval's Theorem.
  - **Pedagogical value:** Rigorous treatment of how filters shape frequency content without requiring FFT.
- Sallen, R. P., & Key, E. L. (1955). "A Practical Method of Designing RC Active Filters," *IRE Transactions on Circuit Theory*, 2(1), 74–85.
  - **Original paper:** Foundational work on the Sallen-Key topology you implement in hardware.
  - **Hardware connection:** Direct lineage between theory and the op-amp filter stages you build.
- Franco, S. (2015). *Design with Operational Amplifiers and Analog Integrated Circuits* (4th ed.). McGraw-Hill.

  - **Chapters 1, 5, 6:** Op-amp fundamentals, frequency response, and active filter design.
  - **Practical focus:** Component selection, power supply design, and real-world issues (offset, slew rate, temperature).

- Williams, A. B., & Taylor, F. J. (2006). *Electronic Filter Design Handbook* (4th ed.). McGraw-Hill.
  - **Chapter 2–3:** Analog bandpass filter design equations and quality factor optimization.
  - **Industrial reference:** Real filter designs for commercial and defense applications.

- IEEE 61000-4-30 (2015). *Testing and Measurement Techniques – Power Quality Measurement Methods*.
  - **Industrial standard:** Used by utilities worldwide to validate harmonic measurement accuracy (e.g., 1% frequency error, 5% amplitude accuracy).
  - **Real-world relevance:** Lab Program 2 (power grid monitoring) operates under these constraints.

- Senturia, S. D. (2001). *Microsensor Design* (MIT Press). Chapter 4: Analog Interface Circuits.
  - **Sensors context:** How analog filter banks interface between sensors and microcontrollers in power-constrained systems.

Software and Tools

- MATLAB (MathWorks). Signal Processing Toolbox, particularly:
  - `fft()`, `tfestimate()` for frequency response measurement.
  - `butter()`, `cheby1()` for digital filter design (for reference/comparison only).
- LTspice (Analog Devices). Free circuit simulation tool for validating filter designs before breadboard assembly.
- Arduino. Real-time data acquisition and signal generation on resource-constrained hardware.

Why This Lab Matters

In 2024, battery-powered IoT devices outnumber grid-powered systems 10:1. Yet most signal processing textbooks assume unlimited compute and power. This lab teaches the **analog-first design philosophy**: extract information continuously at microamp power levels, wake the digital processor only when interesting events occur. That is the engineering reality of modern sensor systems, and it is grounded entirely in Fourier analysis and filter design—the two pillars of this course.